

AGA Loop (Ask-Gap-Action Loop)

The End of Scaling Laws and Why Vibe Coding Never Dies

Abstract

Vibe Coding is often dismissed as an undisciplined development style that collapses under scale.

In our previous formulation, we showed that such collapse is not caused by the absence of actions, but by *false-positive stability* arising from weak observational predicates.

In this paper, we strengthen the theory by introducing **modules as semantic subsets of specification space**, and by formalizing a new inevitable phenomenon: **Drifted Modules**.

A drifted module is a development artifact that is internally consistent with respect to Specification–Test–Implementation (STI) parity, yet no longer anchored to any determinately required intention.

We show that, in systems capable of maintaining STI parity *deterministically at scale*, drifted modules arise as a necessary extremal phase of the Ask–Gap–Action (AGA) loop. Crucially, drift is not a failure of logic or tooling, but a structural consequence of success.

We formalize drift as a Gap abstraction whose resolution cannot be automated. Instead, the unique admissible action is **human annotation**, which reclassifies drifted modules into *Required* or *Rejected*.

We conclude that Vibe Coding never dies: in post-scaling systems, it stabilizes into a human-maintained boundary between semantic completion and intentional meaning.

1. Development as a State Machine

Definition 1 (Development States)

Let D be a non-empty set whose elements $d \in D$ are called **development states**.

A development state represents the totality of artifacts, relations, and meta-information at a given moment of development.
Its internal structure is intentionally left unspecified.

Definition 2 (State Transition)

Let $\rightarrow \subseteq D \times D$ be a binary relation called the **development transition**.

We write $d \rightarrow d'$ if d' is reachable from d by a single development action.
Let $\rightarrow^*$ denote its reflexive–transitive closure.

Each action induces at most one state transition.

2. Specifications, Modules, and Inspection Records

Definition 3 (Specification Space)

Let R be a non-empty set of **specification atoms**.

Elements of R represent minimal, independently meaningful requirements.
No structure is assumed on R .

Definition 4 (Modules)

A **module** is defined as a subset of the specification space:

$$m \subseteq R.$$

Let $M \subseteq \mathcal{P}(R)$ be the set of all modules under consideration.

Intuitively, a module is not a syntactic unit, but a semantic claim:

“This artifact realizes exactly the specifications in m .”

Definition 5 (Artifacts)

Let:

- S be the set of specification artifacts,
- T be the set of test artifacts,
- I be the set of implementation artifacts.

Definition 6 (Inspection Records)

Define the set of **inspection records**:

$$K := S \times T \times I.$$

An element $k = (s, t, i) \in K$ is associated with exactly one module $m \in M$, written $\text{Mod}(k) = m$.

3. Consistency Predicates

Definition 7 (Specification–Test Predicate)

Let

$$P_1 \subseteq D \times K.$$

$P_1(d, k)$ means that, in state d , test t is semantically consistent with specification s .

Definition 8 (Test–Implementation Predicate)

Let

$$P_2 \subseteq D \times K.$$

$P_2(d, k)$ means that, in state d , test t correctly constrains implementation i .

Definition 9 (Local STI Consistency)

$$\text{STI}(d, k) \iff P_1(d, k) \wedge P_2(d, k).$$

Definition 10 (Global STI Parity)

A state d satisfies **STI Parity** if:

$$\text{Parity}(d) \iff \forall k \in K. \text{STI}(d, k).$$

4. Ask–Gap–Action (AGA)

Definition 11 (Gap)

The **Gap** of state d is:

✔ Successfully updated the title and tags ✕

\$\$ G(d)

$\{(k,1)\mid \neg P_1(d,k)\}$
; $\cup$;
 $\{(k,2)\mid \neg P_2(d,k)\}$.
\$\$

Definition 12 (Ask Mapping)

Let $\mathcal{A}$ be the set of actions.

$$\llbracket \text{Ask} \rrbracket: D \rightarrow \mathcal{P}(\text{Gap}) \times \mathcal{P}(\mathcal{A})$$

with

$$\llbracket \text{Ask} \rrbracket(d) = (G(d), \text{Act}(d)),$$

where $\text{Act}(d)$ contains only actions that preserve all currently satisfied predicates.

5. Deterministic STI Maintenance

Assumption 1 (Deterministic Parity Preservation)

The system under consideration satisfies:

For any reachable d and any d' with $d \rightarrow^* d'$,
STI parity violations are deterministically detected and eliminated.

This is a *capability assumption*, not a logical axiom.
Lazy systems empirically satisfy this property at nontrivial scale.

6. Drifted Modules

Definition 13 (Requirement Status)

For each module $m \in M$, define its **requirement status**:

$$\llbracket \text{ReqStatus} \rrbracket(m) \in \{\text{Required}, \text{Rejected}, \text{Unclassified}\}.$$

● Successfully updated the title and tags ✕

Initially, $\backslash(\mathrm{Unclassified})$ is permitted.

Definition 14 (Drifted Module)

A module m is **drifted** in state d iff:

$$\text{Drifted}(d, m) \iff \left(\forall k (\text{Mod}(k) = m \Rightarrow \text{STI}(d, k)) \right) \wedge \text{ReqStatus}(m) = \text{Unclassified}.$$

Thus, a drifted module is:

- internally complete,
- externally intentionless.

Definition 15 (Drift Gap)

Extend the Gap abstraction with:

$$G_{\text{drift}}(d), m \in M \mid \text{Drifted}(d, m),.$$

This is a **semantic Gap**, not a consistency Gap.

7. Actions for Drift

Definition 16 (Annotation Action)

An **annotation action** for module m is:

$$\text{Annotate}(m, \sigma) \quad \text{where} \quad \sigma \in \text{Required}, \text{Rejected}.$$

Annotation actions:

- are human-supplied,
- are irreversible,
- introduce no new technical predicates.

Lemma 1 (Uniqueness of Admissible Actions)

If m is drifted in d , then the only admissible actions affecting m are annotation actions.

Proof.

Any non-annotation action would modify s , t , or i and thus violate STI parity, contradicting admissibility.

□

8. Inevitability of Drift (Extremal Phase)

Theorem 1 (Drift as an Extremal Phase of AGA)

In any system satisfying Deterministic Parity Preservation, for any sufficiently rich exploration of the development space, there exists a reachable state d such that:

$$G(d) = \emptyset \quad \wedge \quad G_{\text{drift}}(d) \neq \emptyset.$$

Proof

1. Deterministic STI maintenance ensures that all consistency gaps are eventually eliminated.
2. Exploration under weak or ambiguous initial intention generates modules realizing nontrivial subsets of R .
3. Completion of STI parity for such modules occurs regardless of whether their originating intention was well-formed.
4. Hence, modules become complete before their requirement status is resolved.

Therefore, drifted modules necessarily arise.

□

9. Interpretation: Post-Scaling Behavior

The emergence of drifted modules is not accidental.

It occurs **only** in systems that:

- preserve consistency at scale,
- eliminate false positives,
- allow exploration without premature intention collapse.

Thus, drift is a *post-scaling phenomenon*.

10. Conclusion

We have shown that:

1. Modules can be rigorously defined as subsets of specification space.
2. Internal completion does not imply intentional necessity.
3. Drifted modules arise inevitably in systems with strong deterministic STI enforcement.
4. Drift is a Gap, but one whose resolution is *inherently human*.
5. Vibe Coding stabilizes into a maintenance loop of annotation, not correction.

Final Statement

Vibe Coding never dies.

In post-scaling systems, it converges to a human-maintained boundary between meaning and completion.